

Transformers

CS221.Q21.KHTN - Xử lý ngôn ngữ tự nhiên

Bùi Ngọc Thiên Thanh - 23521436

Cáp Kim Hải Anh - 23520036

Hoàng Đức Dũng - 23520328

Nguyễn Thái Sơn - 23521356



Khoa Khoa học Máy tính
Trường Đại học Công nghệ thông tin
VNU-HCM

Tháng 4, 2026

Tables of contents

- 1 Giới thiệu
- 2 Cơ chế Attention (Attention Mechanism)
 - Động lực: Giải quyết sự đa nghĩa trong ngữ cảnh
 - Cấu trúc: Causal Self-Attention
- 3 Transformer Blocks
 - Residual connections
 - Feedforward layer
 - Layer norm
- 4 Song song hóa tính toán với ma trận X
 - Song song hóa Attention
 - Cơ chế Masking
 - Multi-Head Attention song song





X

✓

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

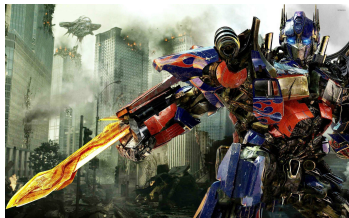
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukasz.kaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

- **Thời kỳ tiền Transformer:** Sự thống trị của RNN và LSTM.
- **Nghịch lý RNN:** Càng đọc dài, càng nhanh quên (*Vanishing Gradient*) và cực kỳ chậm chạp.
- **2017 - Bước ngoặt:** Bài báo “*Attention Is All You Need*” từ Google Brain.
- **Tuyên ngôn gây sốc:** Bỏ hết các kiến trúc tuần tự, chỉ cần cơ chế *Attention* là đủ để hiểu toàn bộ ngôn ngữ thế giới.





X

Attention Is All You Need

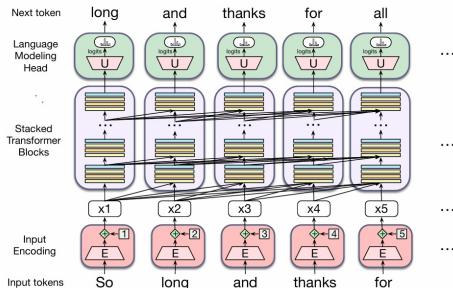
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* [‡] University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
		Illia Polosukhin* [‡] illia.polosukhin@gmail.com	

✓

- **Ưu điểm:** Xử lý **song song** thay vì tuần tự (RNN) → Tối ưu hóa hiệu năng phần cứng (GPU).
 - **Cơ chế Self-Attention:** Giúp mô hình "hiểu" ngữ cảnh toàn cục và mối quan hệ giữa các từ tốt hơn.
- **Kết quả:** Là nền tảng của GPT, BERT và cuộc đua LLMs hiện nay.



Cơ chế Attention (Attention Mechanism)



- **Input (E):** Các vector tĩnh (*static embeddings* như Word2vec) – chưa có ngữ cảnh.
- **Khối Transformer:** Tính toán sự liên quan giữa các từ (theo luồng từ trái sang phải).
- **Output (U):** Các đại diện ẩn (*hidden representations*) đã được “nhúng” ngữ cảnh.



Cơ chế Attention (Attention Mechanism)

Hãy thử đoán đối tượng mà từ **“it”** ám chỉ trong hai câu sau:

Câu 1

*“The chicken didn’t cross the road because **it** was too **tired**.”*

Câu 2

*“The chicken didn’t cross the road because **it** was too **wide**.”*



Cơ chế Attention (Attention Mechanism)

Hãy thử đoán đối tượng mà từ **“it”** ám chỉ trong hai câu sau:

Câu 1

*“The chicken didn’t cross the road because **it** was too **tired**.”*

Câu 2

*“The chicken didn’t cross the road because **it** was too **wide**.”*

→ Ở câu 1: **“it”** → **chicken** (*Trạng thái mệt mỏi*)



Cơ chế Attention (Attention Mechanism)

Hãy thử đoán đối tượng mà từ **“it”** ám chỉ trong hai câu sau:

Câu 1

*“The chicken didn’t cross the road because **it** was too **tired**.”*

Câu 2

*“The chicken didn’t cross the road because **it** was too **wide**.”*

→ Ở câu 1: **“it”** → **chicken** (Trạng thái mệt mỏi)

→ Ở câu 2: **“it”** → **road** (Đặc tính chiều rộng)



Cơ chế Attention (Attention Mechanism)

- **Cách xử lý của Word2Vec (Static Embeddings):**

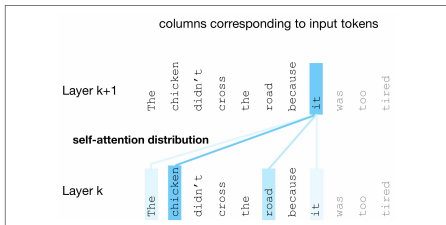
- Luôn gán duy nhất một vector cố định cho từ “it”:
 $V_{it} = [0.21, -0.15, \dots]$ trong cả hai câu.
- **Nhược điểm:** Không phân biệt được đối tượng thực sự. Máy tính coi hai câu có ý nghĩa giống hệt nhau về mặt đại từ.



Cơ chế Attention (Attention Mechanism)

- **Cách xử lý của Attention (Contextual Embeddings):**

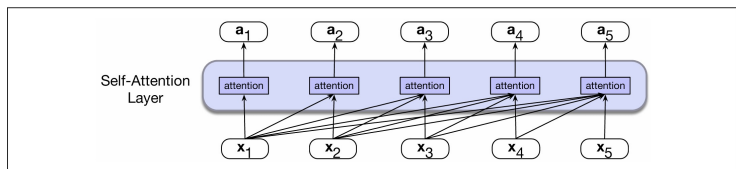
- Thay vì dùng vector cố định, **Attention** cho phép từ “nó” nhìn sang các từ xung quanh.
- Trong câu 1, nó tập trung (*attend*) vào “**chicken**”. Trong câu 2, nó tập trung vào “**road**”.



Word2vec là “tra từ điển”, Attention là “đọc hiểu ngữ cảnh”.



Luồng thông tin trong Causal Self-Attention



Hình 8.4: Luồng thông tin trong lớp Self-Attention (Causal)

- **Cơ chế Causal (Nhân quả):** Tại vị trí i , mô hình chỉ truy cập các token trong quá khứ ($x_1 \dots x_{i-1}$) và chính nó (x_i), tuyệt đối không nhìn thấy tương lai.
- **Xử lý song song:** Toàn bộ các đầu ra ($a_1, \dots, a_n$) được tính toán đồng thời, giúp tối ưu hiệu năng vượt trội so với kiến trúc tuần tự của RNN.



Bản chất của Attention: Tổng có trọng số

- **Ý tưởng chính:** Đầu ra tại một vị trí (a_i) là sự kết hợp thông tin của tất cả các từ đứng trước (x_j).
- **Cơ chế Weighted Sum:** Không phải từ nào cũng quan trọng như nhau. Mỗi từ x_j sẽ đóng góp một lượng α_{ij} vào từ hiện tại.

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j \quad (8.6)$$

Nguyên lý "Similarity" (Độ tương đồng)

- Chúng ta gán trọng số dựa trên mức độ **giống nhau** (similar) giữa từ hiện tại và các từ trong quá khứ.
- Từ nào càng liên quan mật thiết đến ngữ cảnh hiện tại, trọng số α càng lớn.



Quy trình tính toán Attention đơn giản

Để tìm ra đầu ra a_i , mô hình thực hiện 3 bước:

- ➊ **Tính điểm (Score) bằng Dot Product:** Đo độ tương đồng giữa hai vector x_i và x_j .

$$\text{score}(x_i, x_j) = x_i \cdot x_j \quad (8.7)$$

- ➋ **Chuẩn hóa bằng Softmax:** Biến các điểm số thành xác suất (α_{ij}), đảm bảo tổng các trọng số bằng 1.

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j)) \quad \forall j \leq i \quad (8.8)$$

- ➌ **Tổng hợp thông tin:** Nhân trọng số α_{ij} với các vector đầu vào x_j để có kết quả cuối cùng a_i .
- ➔ **Lưu ý:** α_{ii} (tự chú ý chính nó) thường có giá trị cao nhất vì một từ luôn giống chính nó nhất.



Cơ chế Attention Head: Query, Key và Value

Trong thực tế, mỗi input vector x_i sẽ đóng 3 vai trò khác nhau thông qua các ma trận trọng số W^Q , W^K , W^V :

- **Query** ($q_i = x_i W^Q$): Đại diện cho phần tử hiện tại đang đi "hỏi" các phần tử khác.
- **Key** ($k_j = x_j W^K$): Đại diện cho "nhãn" của các phần tử trước đó để so khớp.
- **Value** ($v_j = x_j W^V$): Nội dung thông tin thực sự sẽ được trích xuất.

Sự khác biệt

Thay vì dùng trực tiếp x , việc học các ma trận W giúp mô hình linh hoạt hơn trong việc chọn lọc thông tin nào là quan trọng cho mục đích nào.



Quy trình Scaled Dot-Product Attention

- 1 **Tính Score (Độ tương đồng):** So khớp Query hiện tại với các Key quá khứ.

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad (8.11)$$

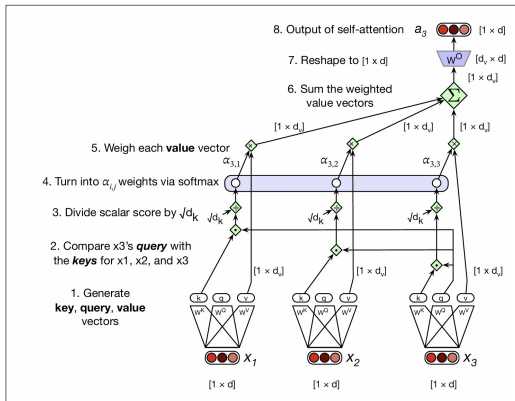
- 2 **Tính Trọng số (α):** Đưa qua Softmax để có tỉ lệ phần trăm.
- 3 **Tính Output (head):** Tổng có trọng số của các vector **Value**.

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} v_j \quad (8.13)$$

Tại sao phải chia cho $\sqrt{d_k}$?

Để tránh việc tích vô hướng quá lớn làm hàm Softmax bị "bão hòa", dẫn đến triệt tiêu gradient (*vanishing gradient*) khi huấn luyện.





Các bước trên sơ đồ:

1. Tạo q, k, v từ x_1, x_2, x_3 .
2. q_3 nhân vô hướng với k_1, k_2, k_3 .
3. Chia $\sqrt{d_k}$ và Softmax $\rightarrow \alpha$.
4. Nhân α với các v_j tương ứng.
5. Qua ma trận W^O để trả về chiều dữ liệu gốc.



Ví dụ toán học: Khởi tạo Ma trận

Xét câu ngắn: “**it is**” với chiều vector $d = 4$.

- **Input Matrix (X):** $X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$ (*it is*)
- Giả sử ta đã tính được $k_1 = [1, 1]$ và $v_1 = [2, 4]$ (cho từ “it”).
- **Tạo Query, Key, Value cho từ “is”** ($x_2 = [0, 1, 0, 1]$):

$$\begin{bmatrix} q_2 \\ k_2 \\ v_2 \end{bmatrix} = x_2 \times \begin{bmatrix} W^Q \\ W^K \\ W^V \end{bmatrix} \rightarrow$$



Ví dụ toán học: Khởi tạo Ma trận

Xét câu ngắn: “it is” với chiều vector $d = 4$.

- **Input Matrix (X):** $X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$ (it is)
- Giả sử ta đã tính được $k_1 = [1, 1]$ và $v_1 = [2, 4]$ (cho từ “it”).
- **Tạo Query, Key, Value cho từ “is”** ($x_2 = [0, 1, 0, 1]$):

$$\begin{bmatrix} q_2 \\ k_2 \\ v_2 \end{bmatrix} = x_2 \times \begin{bmatrix} W^Q \\ W^K \\ W^V \end{bmatrix} \rightarrow \begin{cases} q_2 = [0, 2] & \text{(Query)} \\ k_2 = [1, 1] & \text{(Key)} \\ v_2 = [6, 8] & \text{(Value)} \end{cases}$$



Ví dụ toán học: So khớp và Chuẩn hóa

Bây giờ, từ “is” (x_2) tự đi tìm ngữ cảnh cho mình:

- **1. So khớp & Tỷ lệ:** $S_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$
 - $S_{2,1} = \frac{q_2 \cdot k_1}{\sqrt{2}} = \frac{2}{1.41} = 1.41$
 - $S_{2,2} = \frac{q_2 \cdot k_2}{\sqrt{2}} = \frac{2}{1.41} = 1.41$
- **2. Trọng số:** $\alpha_{ij} = \text{softmax}(S_i)$
 - $\alpha = \text{softmax}([1.41, 1.41]) = [0.5, 0.5]$

3. Kết quả đầu ra: $a_i = \sum \alpha_{ij} v_j$

$$a_2 = 0.5 \times [2, 4] + 0.5 \times [6, 8] = [4, 6] \quad (1)$$

→ Ý nghĩa: a_2 đã tổng hợp 50% thông tin từ chính nó và 50% từ từ đứng trước.



Ví dụ toán học: Phép chiếu đầu ra W^O

Sau khi có kết quả $a_2 = [4, 6]$, bước cuối cùng là đưa nó về lại chiều ban đầu $d = 4$ thông qua ma trận W^O .

- **Ma trận trọng số đầu ra (W^O):** Giả sử $W^O = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$ (kích thước 2×4).

Kết quả cuối cùng của tầng Attention

$$\text{Output}_2 = a_2 \times W^O = [4, 6] \times \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = [4, 6, 4, 6] \quad (2)$$

- ✓ **Kích thước:** $[1 \times 4]$, khớp với kích thước của vector đầu vào x_2 .
- ✓ **Ý nghĩa:** Kết quả này đã sẵn sàng để đi vào lớp Feedforward hoặc lớp Transformer tiếp theo.



Multi-head Attention: Sức mạnh của sự đa nhiệm

- **Vấn đề:** Một cơ chế Attention duy nhất (Single-head) bị giới hạn trong việc tập trung vào nhiều mối quan hệ khác nhau cùng lúc.
- **Giải pháp:** Sử dụng A "đầu" attention (*heads*) hoạt động song song.

Ý tưởng (Intuition)

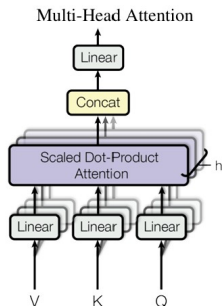
Mỗi "head" sẽ học cách tập trung vào các khía cạnh khác nhau của ngôn ngữ:

- Head 1: Học về mối quan hệ ngữ pháp ngắn hạn.
- Head 2: Học về các đại từ thay thế (như ví dụ "it").
- Head 3: Học về cấu trúc logic của cả đoạn văn.

→ Kết quả là tập hợp các cái nhìn đa chiều về cùng một dữ liệu đầu vào.



Quy trình Multi-head Attention



Sơ đồ Multi-head Attention

- 1 **Linear Projection:** Nhân x với A bộ ma trận (W^{Qc}, W^{Kc}, W^{Vc}) .
- 2 **Parallel Attention:** Tính A đầu Attention độc lập và song song.
- 3 **Concat:** Ghép nối các kết quả:
 $MultiHead = (head_1 \oplus \dots \oplus head_A)$
- 4 **Output Projection:** Nhân với W^O để tổng hợp thông tin về chiều d .



Ví dụ Ma trận: Từ các Head đến Output

Giả sử $d = 512$, số lượng head $A = 8$. Mỗi head xử lý $d_k = 64$.

- **Bước 1: Kết quả từ 8 heads độc lập** Mỗi $head_c$ là một ma trận kích thước $[1 \times 64]$.
- **Bước 2: Phép toán Concatenate ($\oplus$)** Xếp chồng 8 vector cạnh nhau:

$$\underbrace{[h_1]}_{1 \times 64} \oplus \underbrace{[h_2]}_{1 \times 64} \cdots \oplus \underbrace{[h_8]}_{1 \times 64} = \underbrace{[h_1, h_2, \dots, h_8]}_{1 \times 512} \quad (3)$$

- **Bước 3: Ma trận W^O (Trọng số đầu ra)** Nhân kết quả 1×512 với ma trận W^O kích thước 512×512 .

Ý nghĩa kỹ thuật

Phép nhân cuối cùng với W^O cho phép các thông tin từ 8 head khác nhau "tương tác" và lọc ra kết quả tinh túy nhất.



Phân tích thông số: Tại sao chọn $A = 8, d_k = 64$?

Trong kiến trúc Transformer chuẩn (Vaswani et al.):

- **Tổng lực tính toán không đổi:** Việc chia $d = 512$ thành 8 head $d_k = 64$ giúp tổng khối lượng tính toán tương đương với Single-head chiều 512.
- **Tính toán song song tối ưu:** GPU xử lý 8 ma trận nhỏ (64×64) nhanh hơn nhiều so với 1 ma trận lớn (512×512).
- **Sự đa dạng đặc trưng (*Feature Diversity*):** 8 head cho phép mô hình đồng thời chú ý vào 8 vị trí khác nhau trong câu văn.

Công thức vàng

$$d_k = d_v = d/A = 512/8 = 64$$



Transformer Blocks

Khối transformer (Transformer Block) là đơn vị cốt lõi của các mô hình ngôn ngữ lớn. Trong một khối transformers, phép tính self-attention nằm ở cốt lõi, ngoài ra còn có:

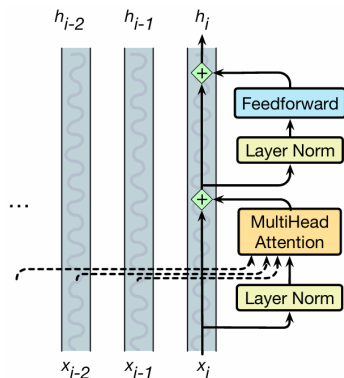
- 1 Residual connections (kết nối thẳng dư).
- 2 Feedforward layer (lớp truyền thẳng).
- 3 Layer norm (lớp chuẩn hoá).



Residual connections

Khái niệm: Coi việc xử lý một token i xuyên qua khối transformers như một luồng duy nhất gồm các vector có d chiều cố định.

- Đầu vào là một vector embedding của từ đầu vào x_i , dần truyền thẳng lên trên và dần dần được đắp thêm thông tin bởi các thành phần khác trong khối.
- Như vậy, vector đầu vào qua một Layer norm và Attention, cộng dồn vào luồng, sau đó, tổng này truyền qua một Layer norm và Feedforward, đầu ra cộng dồn lần nữa, kết quả cuối là h_i .
- Tiền chuẩn hoá (prenorm): dùng Layer norm trước khi qua các lớp Attention và Feedforward.



- Mục đích: Bổ sung thêm thông tin cho vector biểu diễn của token.
- Một mạng nơ-ron fully connected gồm 2 lớp (1 lớp ẩn và 2 ma trận có trọng số), áp dụng hàm kích hoạt ReLU.
- Đặc điểm: Hoạt động độc lập và có trọng số giống hệt nhau cho mọi token i , nhưng khác nhau giữa các lớp transformers.
- Số chiều d_{ff} của lớp ẩn thường lớn hơn nhiều so với số chiều d của mô hình (Ví dụ: $d = 512$ thì $d_{ff} = 2048$).

$$FFN(x_i) = \text{ReLU}(x_i W_1 + b_1) W_2 + b_2 \quad (8.21)$$

Trong đó: W_1 có số chiều $[d \times d_{ff}]$, W_2 có số chiều $[d_{ff} \times d]$



Layer norm

- Mục đích: Giữ các giá trị của một lớp ẩn nằm trong phạm vi nhất định, hỗ trợ quá trình huấn luyện bằng gradient descent, cải thiện hiệu suất.
- Áp dụng cho một vector embedding của một token duy nhất, không áp dụng cho toàn bộ lớp transformers.
- Đầu vào là một vector (số chiều d) và đầu ra là vector đã chuẩn hoá (số chiều d).
- Tính trung bình (μ) và độ lệch chuẩn (σ):

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j \quad (8.22)$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2} \quad (8.23)$$



Layer norm

- Các thành phần của vector được chuẩn hoá:

$$\hat{x} = \frac{(x - \mu)}{\sigma} \quad (8.24)$$

- Kết quả tạo được vector mới $\hat{x}$ có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1.
- Nếu việc chuẩn hoá chỉ dừng lại với $\hat{x}$, đôi khi quá khắt khe và làm mất thông tin quan trọng. Vì vậy, kết quả cuối cùng:

$$\text{LayerNorm}(x) = \gamma \hat{x} + \beta = \gamma \frac{(x - \mu)}{\sigma} + \beta \quad (8.25)$$

Trong đó: γ và β là hai tham số có thể học được trong quá trình huấn luyện, đại diện cho giá trị độ lợi và độ lệch.

- Việc nhân với γ và cộng với β nếu thấy dữ liệu cần độ lệch chuẩn khác 1 và trung bình khác 0 để hoạt động tốt.
- Đặc biệt, nếu $\gamma = \sigma$ và $\beta = \mu$, ta huỷ bỏ việc chuẩn hoá và quay về lại vector x ban đầu.



Tổng kết một khối Transformers

- Hàm tính toán của một khối Transformer cho đầu vào x_i có thể được phân tích thành chuỗi các phương trình (với t là các giá trị trung gian có số chiều $[1 \times d]$):

- Chuẩn hoá trước Attention:

$$t_i^1 = \text{LayerNorm}(x_i) \quad (8.26)$$

- Tính toán Attention (lấy thông tin từ ngữ cảnh):

$$t_i^2 = \text{MultiHeadAttention}(t_i^1, [t_1^1, \dots, t_N^1]) \quad (8.27)$$

- Kết nối thẳng dư lần 1: Cộng kết quả Attention vào dòng chảy ban đầu.

$$t_i^3 = t_i^2 + x_i \quad (8.28)$$

- Chuẩn hoá trước FFN:

$$t_i^4 = \text{LayerNorm}(t_i^3) \quad (8.29)$$

- Đi qua mạng FFN:

$$t_i^5 = \text{FFN}(t_i^4) \quad (8.30)$$

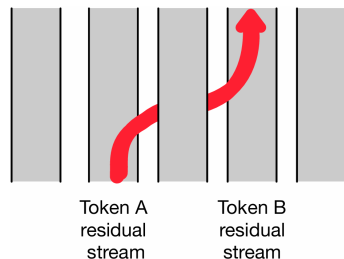
- Kết nối thẳng dư lần 2 (Đầu ra của khối):

$$h_i = t_i^5 + t_i^3 \quad (8.31)$$



Token-mixing

- Thành phần duy nhất lấy thông tin từ các token khác (các dòng chảy khác) là MultiHeadAttention. Một HeadAttention có thể chuyển thông tin từ luồng token A sang luồng của token B.
- Ta gọi đây là thành phần **token-mixing**, vì nó trộn thông tin từ các luồng lân cận và luồng hiện tại.



Khả năng xếp chồng

- Điểm cốt lõi giúp Transformer mạnh mẽ là kích thước đầu vào và đầu ra hoàn toàn khớp nhau. Vector đầu vào x_i có kích thước $[1 \times d]$, và đầu ra h_i cũng có kích thước $[1 \times d]$.
- Nhờ sự đồng nhất này, ta có thể xếp chồng (stack) hàng chục khối lên nhau (từ 12 lớp trong GPT-3-small lên tới 96 lớp trong GPT-3-large).
- Ở các khối transformers đầu tiên, luồng thẳng dư mang ý nghĩa biểu diễn cho token hiện tại, nhưng khi đến các khối transformers cao nhất, luồng thẳng dư này thường mang ý nghĩa biểu diễn cho token tiếp theo.
- Vì ta dùng **prenorm**, dòng chảy đi qua khối cuối cùng chưa được chuẩn hóa ở đầu ra. Do đó, cần thêm một lớp Layer Norm duy nhất ở ngay sau khối Transformer cuối cùng, trước khi đi vào Language Modeling Head.



Song song hóa tính toán với ma trận X

- Các mô tả trước đây tập trung vào việc tính toán cho một token đơn lẻ tại một thời điểm i .
- **Đặc điểm cốt lõi:** Phép tính attention và các thành phần trong khối Transformer cho mỗi token là độc lập với nhau.
- **Giải pháp:** Parallelization (Song song hóa) toàn bộ quá trình bằng cách tận dụng các thư viện nhân ma trận hiệu năng cao.
- Gom các vector embedding của N token vào một ma trận duy nhất X kích thước $[N \times d]$.
- Quy mô ngữ cảnh (N) thường từ 1K đến 32K tokens, thậm chí lên đến hàng triệu.



Song song hóa Attention

Thay vì tính từng vector q_i, k_i, v_i , ta tính toán đồng thời cho toàn bộ chuỗi X :

- **Tạo các ma trận Q, K, V:**

$$Q = XW^Q; \quad K = XW^K; \quad V = XW^V \quad (8.32)$$

- **Kích thước:** $Q, K \in \mathbb{R}^{N \times d_k}$ và $V \in \mathbb{R}^{N \times d_v}$.
- **Tính toán tương đồng (Score Matrix):** Nhân Q và K^T để tạo ra ma trận $[N \times N]$ chứa tất cả các cặp so sánh $q_i \cdot k_j$.

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
N				



Cơ chế Masking (Che giấu tương lai)

- **Vấn đề:** Trong mô hình ngôn ngữ causal, việc đoán từ tiếp theo không được phép biết trước các từ đứng sau.
- **Giải pháp:** Sử dụng hàm *mask*. Các phần tử ở tam giác trên của ma trận QK^T được đặt thành $-\infty$.
- Khi qua hàm Softmax, các giá trị $-\infty$ này sẽ trở thành 0, loại bỏ hoàn toàn ảnh hưởng từ các token tương lai.

N

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

Hình 8.10: Ma trận QK^T sau khi áp dụng mask.



Công thức Self-Attention song song

Toàn bộ bước self-attention cho một chuỗi N token của một head được rút gọn thành:

$$head = \text{softmax} \left(\text{mask} \left(\frac{QK^T}{\sqrt{d_k}} \right) \right) V \quad (8.33)$$

- Kết quả là một ma trận kích thước $[N \times d_v]$, đại diện cho các embedding ngữ cảnh của mọi token trong chuỗi.
- Sau đó, kết quả được chiếu qua ma trận W^O để thu được ma trận đầu ra A kích thước $[N \times d]$:

$$A = headW^O \quad (8.34)$$



Multi-Head Attention song song

- **Thực hiện:** Tính song song A heads, mỗi head có bộ ma trận W^Q, W^K, W^V riêng.
- **Kết hợp:** Các ma trận đầu ra $[N \times d_v]$ của từng head được nối (concatenate) lại thành ma trận $[N \times A d_v]$.
- **Chiều cuối cùng:** Sử dụng $W^O \in \mathbb{R}^{A d_v \times d}$ để đưa về kích thước ban đầu $[N \times d]$.

$$\text{MultiHeadAttention}(X) = (\text{head}_1 \oplus \text{head}_2 \cdots \oplus \text{head}_A) W^O \quad (8.37)$$

- **Lưu ý:** Attention có độ phức tạp **quadratic** ($O(N^2)$) theo chiều dài chuỗi vì phải tính tích vô hướng giữa mọi cặp token.



Tổng hợp tính toán song song trong một lớp

Hàm tính toán song song cho một lớp gồm N khối Transformer được biểu diễn như sau (với X là đầu vào của lớp):

$$O = X + \text{MultiHeadAttention}(\text{LayerNorm}(X)) \quad (8.38)$$

$$H = O + \text{FFN}(\text{LayerNorm}(O)) \quad (8.39)$$

- Ở đây, $\text{FFN}(O)$ có nghĩa là cùng một mạng Feedforward được áp dụng song song cho từng hàng (từng token) trong ma trận O .
- Tương tự, LayerNorm cũng được thực hiện song song cho N token.
- Kích thước được bảo toàn: Cả đầu vào X và đầu ra H đều có dạng $[N \times d]$.

